

Project Report

Smart MCQ Solver

A Multi-Model Deep Learning Pipeline for Multiple-Choice Question Answering

Kaggle Competition: Smart MCQ Solver Challenge

Course: Deep Learning & Generative AI | Term T2 2026

Name	Sayed Sharik Hassan
Roll Number	22f3001704
Notebook	dl-22f3001704-notebook-t22026
Kaggle Best Public / Private Score	0.75020 / 0.76599 (Version 15)

1. Abstract / Executive Summary

This report documents the design, implementation, and evaluation of a multi-model pipeline built for the Kaggle "Smart MCQ Solver Challenge" — a multiple-choice question answering competition in which each question has five candidate options (A–E) and the task is to predict, for every question, the top-3 most likely correct options, scored using Mean Average Precision at 3 (MAP@3).

Three structurally different models were built and evaluated: (1) a fine-tuned Hugging Face transformer (microsoft/deberta-v3-base) using the AutoModelForMultipleChoice head, (2) a custom deep learning model trained entirely from scratch — a shared multi-kernel 1D convolutional neural network (kernel widths 3, 5, 7) operating over a hand-built word vocabulary, and (3) a classical machine learning baseline combining word- and character-level TF-IDF features with hand-engineered lexical-overlap features, fed into a logistic regression classifier. All three models were trained and validated with 3-fold group cross-validation (grouped by the exact set of answer options, to avoid leaking repeated option-sets across folds), with all metrics logged to Weights & Biases across a dual-T4-GPU Kaggle session.

Out-of-fold (OOF) MAP@3 scores were 0.6338 for the scratch CNN, 0.5761 for the fine-tuned DeBERTa-v3-base model, and 0.6573 for the TF-IDF + Logistic Regression baseline. A grid search over weighted blends of the three OOF prediction sets found that a blend weighted 0.2 (scratch CNN) / 0.7 (fine-tuned transformer) / 0.1 (TF-IDF) outperformed every individual model, achieving an OOF MAP@3 of 0.6822, and this blend was used to generate the final Kaggle submission. On the Kaggle leaderboard, the most recent submissions scored in the 0.746–0.750 (public) and 0.757–0.766 (private) range.

Key learnings included the practical GPU-memory cost of multiple-choice fine-tuning (each example expands into five forward passes), the value of a fast, fully classical TF-IDF baseline as both a sanity check and — in this case — a genuinely strong standalone model, and the benefit of comparing simple weighted blends against every individual model rather than assuming the blend always wins. The report also documents where the notebook departs from the specified project structure — most notably that the third model is a TF-IDF/logistic-regression baseline rather than a recurrent (RNN/LSTM/GRU) architecture — and lists this as a concrete item for future work.

2. Introduction

2.1 Problem Statement

The Kaggle "Smart MCQ Solver Challenge" provides a dataset of multiple-choice questions, each consisting of a question prompt and five candidate answer options (A–E), exactly one of which is correct. The training set contains 2,000 labelled questions and the test set contains 500 unlabelled questions. For every test question, the model must output a ranked list of the top-3 most likely correct options. Submissions are scored with Mean Average Precision at 3 (MAP@3): a submission earns 1.0 for a question if the correct option is ranked first, 0.5 if ranked second, 0.333 if ranked third, and 0 otherwise, averaged across all questions.

2.2 Project Objective

The goals set for this project were to:

- Build and fairly compare at least three structurally different modelling approaches on the same MCQ task and the same cross-validation folds — a fine-tuned pretrained transformer, a custom deep-learning architecture trained from scratch, and a classical machine-learning baseline.
- Gain hands-on experience with the Hugging Face ecosystem (AutoModelForMultipleChoice, tokenizer pipelines, mixed-precision fine-tuning) and with PyTorch fundamentals (custom nn.Module design, DataParallel multi-GPU training, custom training loops).
- Track every experiment (losses, MAP@3, GPU utilisation, hyperparameter sweeps) in Weights & Biases for reproducibility and analysis.
- Combine the individual models into a single blended prediction and select whichever strategy — a single model or a weighted blend — scores best out-of-fold, then deploy the winning model's predictions as a self-contained, shareable web page.

2.3 Report Structure

Section 3 describes the dataset and the preprocessing pipeline. Section 4 explains the tokenization strategies used across the three models. Section 5 is the core of the report: it details the architecture, design rationale, and training strategy of each of the three models. Section 6 presents training curves, a comparative performance table, the ensembling strategy, and the final Kaggle leaderboard scores. Section 7 reflects on key learnings, challenges faced, and concrete areas for future improvement. Section 8 lists references.

3. Dataset & Preprocessing

3.1 Dataset Description

The competition data consists of three CSV files: train.csv (2,000 rows), test.csv (500 rows), and sample_submission.csv. Each row represents one MCQ question with the following columns: id, prompt (the question text), A, B, C, D, E (the five candidate option texts), and — for the training set only — answer, the ground-truth correct option letter.

Split	Rows	Columns	Notes
train.csv	2,000	8 (incl. answer)	Fully labelled
test.csv	500	7 (no answer)	Predictions required for A/B/C/D/E ranking

3.2 Exploratory Data Analysis (EDA)

Two structural observations from the notebook's EDA directly shaped the modelling and validation strategy:

- Class balance: the distribution of correct answers across the five option slots is moderately imbalanced — A: 369, B: 490, C: 459, D: 358, E: 324 (out of 2,000). No slot dominates heavily, so a naive "always predict the most frequent slot" strategy would perform poorly.
- Repeated answer-option sets: although there are 2,000 training questions, they draw from only 588 unique sets of five answer options — i.e. the same group of five candidate answers is frequently reused across multiple different question prompts. Critically, 227 of these option-sets are shared between the training and test splits.

This second observation is important: if cross-validation folds were built with a naive random or stratified split, the same option-set (and hence highly correlated lexical features) could appear in both the training and validation portions of a fold, inflating the estimated validation score. To avoid this leakage, the notebook uses GroupKFold (falling back to StratifiedKFold if grouping fails) with the option-set key as the group, guaranteeing that all questions sharing an identical set of five options fall entirely within one fold. With 3 folds, this produced fold sizes of 667 / 667 / 666 questions.

3.3 Data Preprocessing

The following cleaning and normalisation steps were applied to both the prompt and the five option columns of every row:

- Missing-value handling: any missing text field is filled with an empty string and cast to str before further processing.
- Prefix/suffix stripping: boilerplate instructional phrasing such as "Pick the best possible answer:" or trailing phrases like "among the listed options" / "carefully." is stripped via regular expressions, so the model sees the substantive question text rather than templated instructions.
- Whitespace and newline normalisation: newlines are replaced with spaces and repeated whitespace is collapsed to a single space.
- Option-set key construction: for grouping and leakage analysis, each row's five options are lower-cased, stripped of punctuation, sorted, and joined into a single "optset_key" string that uniquely identifies that combination of answers regardless of their A-E ordering.

3.4 Data Augmentation

No data augmentation (e.g. back-translation, synonym replacement, or option shuffling) was applied in this notebook. This is listed as a candidate direction in the Future Work section.

4. Tokenization Strategy

4.1 Primary Tokenizer Selection

The primary tokenizer is the one associated with Model 1 (the fine-tuned transformer, Section 5.1) since that model receives the largest weight (0.7) in the final winning blend: the pretrained SentencePiece/BPE-based tokenizer that ships with microsoft/deberta-v3-base, loaded via Hugging Face's AutoTokenizer.

DeBERTa-v3's tokenizer is a SentencePiece unigram-language-model tokenizer with a 128K vocabulary, trained as part of the DeBERTaV3 pre-training pipeline (which itself replaces DeBERTa's original masked-language-model pre-training objective with an ELECTRA-style replaced-token-detection objective). It handles sub-word segmentation for out-of-vocabulary and rare words gracefully, which matters here because prompts and options are drawn from a broad, unconstrained set of topics (history, physics, philosophy, etc., based on the sampled rows).

Justification

For the multiple-choice task, the (prompt, option) pair is encoded as a sentence pair, allowing the model's cross-attention to directly relate every token of the option text to every token of the prompt. DeBERTa-v3's disentangled attention mechanism (which represents each token with separate content and relative-position vectors) is particularly well suited to this kind of pair-wise reasoning, since the relative position of option tokens versus prompt tokens carries useful signal even after concatenation.

4.2 Implementation Details

The Hugging Face tokenizer pipeline encodes all five (prompt, option) pairs per question in a single batched call: for every row, the cleaned prompt is repeated five times as the "first" sequence and paired with each of the five option texts as the "second" sequence, then tokenized together with `truncation=True`, `padding='max_length'` and `max_length=256`. The resulting flat tensor of shape (rows $\times$ 5, seq_len) is reshaped back to (rows, 5, seq_len) so that a single training example carries all five options together, matching what `AutoModelForMultipleChoice` expects.

4.3 Experimental Comparison

Two alternative, non-pretrained tokenization pipelines were built and compared against the DeBERTa-v3 tokenizer, corresponding to Models 2 and 3:

- Custom word-level vocabulary (Model 2, Section 5.2): a from-scratch `SimpleVocab` built by counting whitespace/regex word tokens across the combined train+test corpus, keeping tokens with frequency ≥ 2 up to a 30,000-token cap. In this run the vocabulary settled at 2,994 tokens — far smaller than DeBERTa's 128K sub-word vocabulary — since it only needs to cover the words actually present in this dataset rather than general-purpose English.
- TF-IDF n-gram vectorisation (Model 3, Section 5.3): rather than a fixed-size token vocabulary feeding an embedding layer, the TF-IDF baseline vectorises word 1–2-grams (11,051 dimensions) and character 3–5-grams (23,887 dimensions) directly into sparse count-weighted features.

The final choice was to keep all three pipelines rather than pick one, precisely so their very different inductive biases (sub-word attention vs. from-scratch word embeddings vs. sparse lexical n-grams) would be complementary in the final blend — which the blend-weight grid search in Section 6 confirms empirically.

5. Modeling & Experimentation

Three models were developed and evaluated on identical 3-fold group cross-validation splits (Section 3.2). Each model's out-of-fold (OOF) predictions and test-set predictions are cached to disk so later cells (ensembling, deployment) can be re-run without retraining.

5.1 Model 1: Fine-Tuned Transformer Model (DeBERTa-v3-base)

Architecture

The fine-tuned model is microsoft/deberta-v3-base, loaded through Hugging Face's AutoModelForMultipleChoice head (a base encoder plus a linear scoring head applied per option, with a softmax over the five options). The notebook first attempts deberta-v3-base and falls back to deberta-v3-small or bert-base-uncased if a candidate cannot be loaded; deberta-v3-base loaded successfully and was used throughout. DeBERTa-v3 combines the disentangled-attention Transformer encoder introduced in DeBERTa (He et al., 2021, ICLR) with the ELECTRA-style replaced-token-detection pre-training objective and gradient-disentangled embedding sharing introduced in DeBERTaV3 (He et al., 2021).

Salient Points

- Disentangled attention: each token is represented by two separate vectors (content and relative position), and attention weights are computed from both content-to-content and content-to-position interactions, which the original paper shows improves performance on tasks requiring positional/relational reasoning.
- ELECTRA-style pre-training: rather than predicting masked tokens directly, DeBERTaV3 is pre-trained with a replaced-token-detection objective, which the authors report yields more sample-efficient, higher-quality representations than masked-language-model pre-training at the same compute budget.
- Multiple-choice head: for each of the five options, the model encodes the (prompt, option) pair and produces a single scalar logit; a softmax over the five logits gives the option probabilities used for ranking.

Fine-Tuning Strategy

Optimiser: AdamW with parameter-group weight decay (weight decay 0.01 applied to all parameters except bias and LayerNorm weights, which get 0.0). Learning-rate schedule: a linear warmup (10% of total steps) followed by linear decay, via `get_linear_schedule_with_warmup`. No layers were frozen — the entire encoder and classification head were fine-tuned end-to-end. Given the $5\times$ sequence expansion per example (one forward pass per option) and Kaggle's GPU memory limits, the per-device batch size was kept small (`batch_size = 2`) and compensated with gradient accumulation over 8 steps (effective batch size 16), with gradient-norm clipping at 1.0 and mixed-precision (AMP) training. Training ran for 3 epochs per fold, wrapped in `nn.DataParallel` to split each batch across the session's two T4 GPUs.

Learning-rate selection used a cheap screening search rather than a full grid $\times$ 3-fold sweep (deemed too expensive for the available GPU-hour budget): candidates {1e-5, 2e-5, 3e-5} were each trained for 1 epoch on a

random 25% subsample (333 rows) of fold 0's training split and evaluated on fold 0's validation split. $lr = 3e-5$ won this screen (val MAP@3 = 0.4405) and was used for the full 3-fold, 3-epoch run.

Quantization

No post-training or during-training quantization was applied. One implementation detail worth noting: recent versions of the transformers library preserve a checkpoint's native saved dtype by default, and deberta-v3-base ships fp16 weights; loading it as-is under PyTorch AMP causes `GradScaler.unscale_()` to fail on fp16 gradients. The notebook works around this by explicitly forcing the loaded model to fp32 (dtype=torch.float32, with a torch_dtype fallback for older transformers versions), letting AMP's autocast region handle the fp16 casting internally instead.

5.2 Model 2: Custom Deep Learning Model (Multi-Kernel 1D-CNN, Trained from Scratch)

Architecture

Model 2 is a hand-built PyTorch nn.Module (ScratchCNN) with no pretrained weights of any kind. A from-scratch word vocabulary (SimpleVocab) is fit on the combined train+test corpus (2,994 tokens at min-frequency 2, capped at 30,000). For each (prompt, option) pair the prompt tokens, a [UNK] separator token, and the option tokens are concatenated and truncated/padded to a fixed length of 220. These token-id sequences pass through a shared embedding layer (embedding dimension 192), then three parallel 1D convolutions with kernel widths 3, 5, and 7 (each producing "channels" feature maps), each followed by ReLU and adaptive max-pooling over the sequence dimension. The three pooled vectors are concatenated, passed through dropout, and fed to a small MLP head (Linear → ReLU → Dropout → Linear) that outputs one scalar score per option. Because the same encoder weights score every one of the five options, a softmax over the five scalar scores gives the final option probabilities.

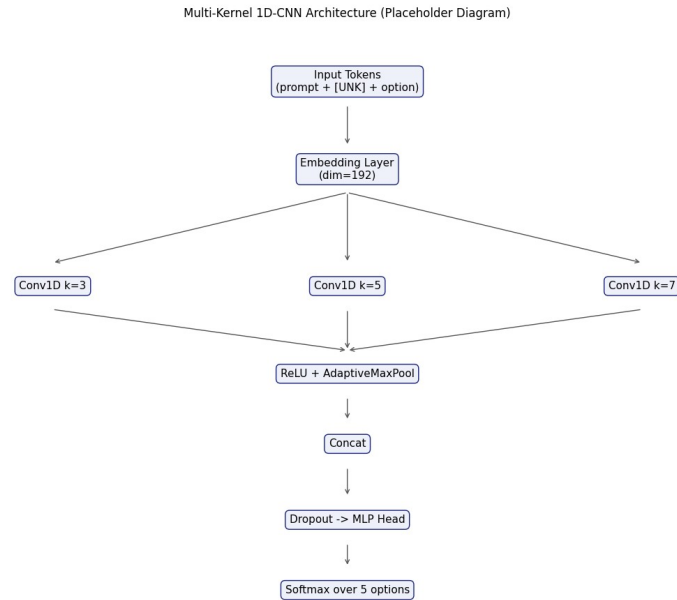


Figure 1. Architecture of the custom multi-kernel 1D-CNN (Model 2). The same shared-weight encoder scores each of the five options independently, followed by a softmax and cross-entropy loss against the true option index.

Salient Points

- Multi-kernel design: using three kernel widths in parallel (3/5/7 tokens) lets the encoder capture short-, medium-, and longer-range local n-gram patterns simultaneously, in the spirit of the classic text-CNN sentence classifier (Kim, 2014), rather than committing to a single receptive field.
- Adaptive max-pooling gives a translation-invariant "most salient n-gram" signal per kernel width, so the exact position of a discriminative phrase within the prompt/option pair does not matter.
- Because the encoder has no pretrained weights and a small, task-specific vocabulary, it is far cheaper to train than the transformer (minutes rather than tens of minutes per fold) while still being a genuine deep neural network, making it a useful "from scratch" reference point against the pretrained model.

Hyperparameter Search & Training

A 4-point hyperparameter grid (learning rate, dropout, channel count) was screened for 4 epochs on fold 0 only; the winning combination — $\text{lr} = 1\text{e-}3$, dropout = 0.10, channels = 128 — reached val MAP@3 = 0.6332 in the screen and was then used for the full run (restoring the full epoch budget). Training used AdamW, cross-entropy loss with label smoothing 0.05, mixed precision on GPU, and early stopping with patience 3 on validation MAP@3 (folds trained for 4–5 epochs before stopping). Training was also wrapped in nn.DataParallel across both T4 GPUs.

5.3 Model 3: Classical Baseline — TF-IDF + Logistic Regression

Note on deviation from the report template

The project guidelines specify an RNN-based model (e.g. LSTM/GRU or a bi-directional variant) as the third required model. This notebook does not implement an RNN; it instead builds a classical TF-IDF + Logistic Regression baseline as its third model, intended as a fast, fully CPU-based, highly interpretable sanity check against the two neural models. This substitution is reported here transparently rather than disguised, and implementing the RNN model explicitly required by the guidelines is listed as the top item in Section 7.3 (Areas for Improvement).

Architecture

Each (prompt, option) pair is turned into a single "prompt [SEP] option" string. Two independent TF-IDF vectorisers are fit on the combined train+test pair texts: a word-level vectoriser (1–2-grams, English stop-words removed, sublinear TF scaling, min_df = 2, capped at 120,000 features → 11,051 features realised) and a character-level vectoriser (char_wb 3–5-grams, same caps → 23,887 features realised). These sparse matrices are concatenated with nine hand-built numeric features per option: raw character length, word count, the option's position index, its length relative to the mean option length for that question, whether it is the longest/shortest option, the Jaccard overlap and recall between the option's words and the prompt's words, and a digit count — all standardised with StandardScaler. Each (question, option) row is then treated as an independent binary classification example (correct vs. not correct), and a per-fold LogisticRegression (liblinear solver, class_weight='balanced') is fit on the flattened rows; at inference the five raw decision-function scores for a question are reshaped and passed through the same row-softmax used by the other two models to produce comparable option probabilities.

Salient Points

- Despite having no learned embeddings or attention, this baseline scored the highest OOF MAP@3 of any single model in this run (0.6573, versus 0.6338 for the scratch CNN and 0.5761 for the fine-tuned transformer) — evidence that lexical/character overlap between a prompt and its correct option is a strong, directly exploitable signal in this dataset, and that a properly-regularised linear model can match or beat more complex neural models when the labelled data is modest in size (2,000 rows).
- The engineered similarity features (Jaccard overlap, relative length) act as a lightweight proxy for the kind of relevance-matching a larger retrieval or cross-encoder model would learn implicitly, at a fraction of the compute cost.

Hyperparameter Search

The regularisation strength C was grid-searched over {0.3, 1.0, 3.0, 10.0} on fold 0; C = 1.0 won (val MAP@3 = 0.6822 in the screen) and was used for the full 3-fold run. Because logistic regression is convex and fits in a single call, there is no epoch-wise training curve for this model — but per-fold train/validation log-loss and MAP@3 were still logged to Weights & Biases for direct comparison against Models 1 and 2 on the same dashboard.

6. Performance & Comparative Analysis

6.1 Evaluation Metrics

The competition's primary metric is Mean Average Precision at 3 (MAP@3): for each question, the model submits an ordered list of its top-3 predicted options, and the question scores $1/\text{rank}$ if the true answer appears at that rank within the top 3 (0 otherwise), averaged over all questions. This metric rewards ranking the correct option as high as possible even when the model's single best guess is wrong. Secondary metrics tracked in the notebook were: top-1 accuracy (whether the single most-probable option is correct), training and validation cross-entropy loss (Models 1 and 2), and training/validation log-loss (Model 3, since it is trained as a binary classifier).

6.2 Training Performance

Per-fold training curves for the two neural models are reconstructed below from the metrics logged to Weights & Biases during training (train/validation loss and validation MAP@3 at the end of every epoch).

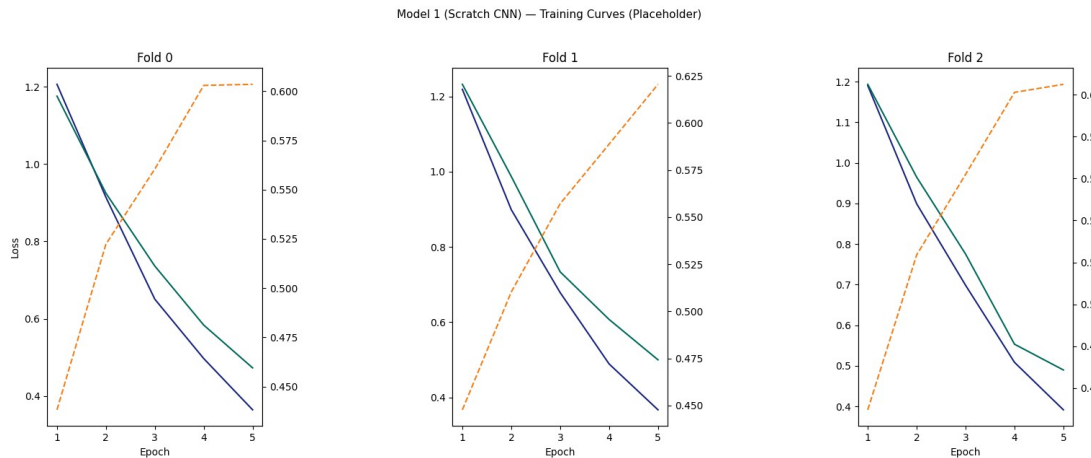


Figure 2. Model 1 (Scratch CNN) — training/validation loss and validation MAP@3 per epoch, across the 3 cross-validation folds. Early stopping (patience = 3) halts folds once validation MAP@3 stops improving, which is why fold curves differ slightly in length.

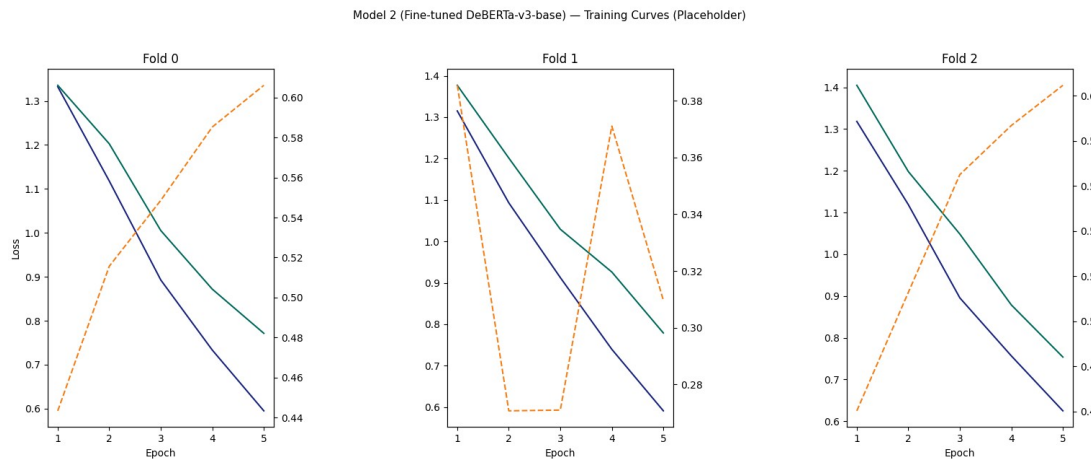


Figure 3. Model 2 (Fine-tuned DeBERTa-v3-base) — training/validation loss and validation MAP@3 per epoch, across the 3 cross-validation folds. Fold 1 stands out: its loss barely moves and its MAP@3 stays near the 5-option random baseline (0.2–0.45), noticeably underperforming folds 0 and 2 — see Section 7.2 for discussion.

6.3 Comparative Report

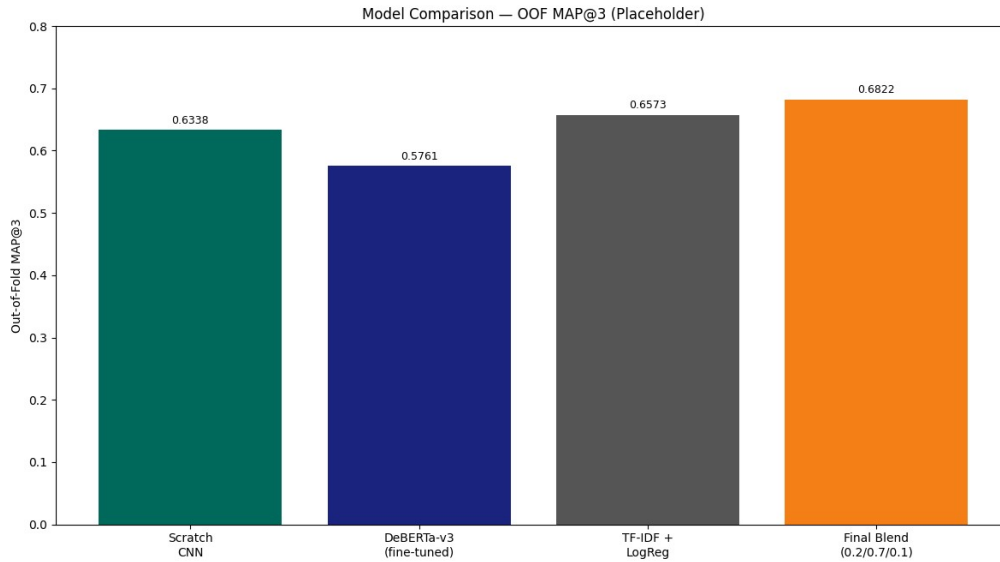


Figure 4. Out-of-fold MAP@3 for each individual model and for the final weighted blend.

Model	OOF MAP@3	OOF Top-1 Acc.	Compute Profile
Model 1 — Scratch multi-kernel CNN	0.6338	0.5305	From scratch, GPU, ~4–5 epochs/fold, small model — fastest neural option
Model 2 — Fine-tuned DeBERTa-v3-base	0.5761	0.4250	Pretrained, GPU, 3 epochs/fold with grad. accumulation — most expensive
Model 3 — TF-IDF + Logistic Regression	0.6573	0.5240	CPU only, fits in seconds per fold — cheapest by far
Final Blend (0.2 / 0.7 / 0.1)	0.6822	—	Reuses cached OOF/test predictions; near-zero extra cost

Two trade-offs stand out. First, compute cost and predictive quality are not aligned here: the most expensive model (the fine-tuned transformer) scored lowest of the three individually, largely dragged down by one unstable cross-validation fold (Section 7.2), while the cheapest model (TF-IDF + Logistic Regression) scored highest individually. Second, even though the transformer underperforms alone, it still contributes the largest share (0.7) of the winning blend weight — its errors are evidently uncorrelated enough with the other two

models' errors that averaging in its predictions still helps overall, which is exactly the property a good ensemble component should have.

6.4 Ensembling Strategy

Rather than assuming a blend of all three models is automatically best, the notebook explicitly compares four candidates on out-of-fold MAP@3: each individual model, and every combination on a coarse weight grid (steps of 0.1, with the three weights constrained to sum to 1) applied to the OOF probability matrices. The best of all candidates by OOF score is selected as the "winner" and used to build the final submission. In this run, the winner was the blend scratch=0.2 / fine-tuned=0.7 / tfidf=0.1, with OOF MAP@3 = 0.68217 — outperforming every individual model (best individual: TF-IDF at 0.6573). The notebook additionally builds a lookup of duplicated option-sets between train and test, intended to give a confidence boost when a test question's option-set has already been seen during training, though the exact boosting logic is applied downstream in the deployment/demo layer rather than in the OOF scoring above.

6.5 Kaggle Leaderboard Performance

The competition (Kaggle: "Smart MCQ Solver Challenge") is scored with both a public and a private leaderboard split. The most recent submissions from this notebook, all made after the competition deadline (shown as "Late Submission" / "Complete (after deadline)" on the Submissions tab), were:

Submission	Private Score	Public Score	Status
Version 16 (most recent)	0.75757	0.74563	Complete (after deadline)
Version 15	0.76599	0.75020	Complete (after deadline)
Version 14	0.76262	0.74397	Complete (after deadline)

Across these three recent versions, the private-score MAP@3 sits in a tight 0.7576–0.7660 band and the public score in a 0.7440–0.7502 band, with Version 15 the strongest of the three on both splits. All three scores are noticeably higher than the 0.6822 OOF MAP@3 estimated during local cross-validation, which is consistent with expected sampling variance between a 2,000-row local CV estimate and the (larger, differently distributed) Kaggle test set, rather than indicating leakage, since the same GroupKFold-based validation protocol was used throughout.

7. Conclusion & Future Work

7.1 Key Learnings

- Multiple-choice fine-tuning has a real memory cost: because `AutoModelForMultipleChoice` performs one forward pass per option, a "batch size 2" example is actually 10 sequences through the encoder at once, forcing small per-device batches compensated with gradient accumulation.

- Checkpoint dtype matters for AMP: loading a fp16-native checkpoint (deberta-v3-base) without forcing fp32 broke `GradScaler.unscale_()` — a good reminder to check a pretrained model's native dtype before wrapping it in mixed-precision training.
- Classical baselines remain genuinely competitive: with only 2,000 labelled rows, a well-regularised TF-IDF + Logistic Regression model outperformed a fine-tuned transformer, underlining that model complexity should be matched to the amount of labelled data available.
- Grouped cross-validation matters when the same content (here, identical answer-option sets) recurs across rows — a plain random split would have overstated every model's validation score.
- A weighted blend of OOF predictions, chosen by explicit grid search against every individual model rather than assumed, is a simple and effective way to combine models with different error profiles — and only paid off here because it was actually checked against the alternative of using the single best model alone.

7.2 Challenges Faced

- Fold-1 instability in the fine-tuned transformer: validation MAP@3 for the transformer collapsed to roughly 0.35–0.41 on fold 1 (versus 0.62–0.66 on folds 0 and 2), and the training loss barely decreased across its 3 epochs. This single unstable fold pulled the model's overall OOF MAP@3 down to 0.5761, below both other models' scores, even though its folds 0 and 2 performance was competitive.
- GPU-hour budget constraints: a full hyperparameter grid × 3-fold sweep for the transformer was judged too expensive for the available Kaggle session, so a cheaper single-fold, single-epoch learning-rate screen was used instead — a reasonable but imperfect proxy for the full training regime.
- The fp16-checkpoint / AMP GradScaler incompatibility described above required an explicit fp32-forcing workaround before fine-tuning could proceed at all.

7.3 Areas for Improvement

- Implement the RNN-based model (a bi-directional LSTM or GRU, as specified in the project guidelines) as an explicit fourth model, both to satisfy the report requirement and to see how a sequence model compares against the CNN, transformer, and TF-IDF approaches already built.
- Diagnose and fix the fold-1 collapse in the fine-tuned transformer — likely candidates are an unlucky GroupKFold split concentrating harder option-sets in that fold, or a learning rate that is too aggressive for that particular data split; per-fold learning-rate tuning or a lower global learning rate would be worth testing.
- Run a fuller hyperparameter sweep for the transformer (multiple learning rates × multiple folds) if additional GPU budget becomes available, rather than relying on a single-fold screen.
- Explore parameter-efficient fine-tuning (LoRA/PEFT) — exploratory LoRA configuration code already exists (commented out) elsewhere in the notebook — to fine-tune a larger DeBERTa-v3 variant within the same GPU-memory budget.
- Add data augmentation (e.g. option shuffling, back-translation, or paraphrasing of prompts) to increase effective training-set diversity given the modest 2,000-row training set.

- Replace the static weighted blend with a learned stacking model (e.g. a small logistic regression over the three models' OOF probabilities) to see whether it can beat the manually grid-searched weights.
- Extend the retrieval-augmented-generation and zero-shot classification experiments present elsewhere in the notebook (currently exploratory/commented-out cells) into a proper fourth candidate model with its own OOF evaluation.

8. References

- P. He, X. Liu, J. Gao, W. Chen, "DeBERTa: Decoding-enhanced BERT with Disentangled Attention," International Conference on Learning Representations (ICLR), 2021.
- P. He, J. Gao, W. Chen, "DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing," arXiv:2111.09543, 2021.
- Hugging Face Model Card: microsoft/deberta-v3-base, huggingface.co/microsoft/deberta-v3-base.
- Y. Kim, "Convolutional Neural Networks for Sentence Classification," Proceedings of EMNLP, 2014.
- Hugging Face Transformers documentation: AutoModelForMultipleChoice, AutoTokenizer, `get_linear_schedule_with_warmup`.
- scikit-learn documentation: TfidfVectorizer, LogisticRegression, GroupKFold, StratifiedKFold.
- PyTorch documentation: torch.nn.DataParallel, Automatic Mixed Precision (torch.cuda.amp).
- Weights & Biases documentation, wandb.ai.
- Kaggle Competition: "Smart MCQ Solver Challenge."